

Capítulo 6. Organizando nuestros programas mediante funciones y procedimientos

Caso de estudio

Evolucionando el Sistema de Control de Parqueos

Introducción

En el capítulo anterior desarrollamos un programa completo para calcular el cobro de un parqueo. El programa resolvía correctamente el problema planteado y nos permitió integrar variables, operadores y estructuras de control.

Ahora el programa funciona, pero ha crecido. Cuando un programa aumenta de tamaño también aumenta la dificultad para leerlo, comprenderlo y modificarlo.

En el desarrollo profesional del software es poco común escribir todo dentro de `main()`. En su lugar, el programa se divide en pequeños módulos donde cada uno realiza una única responsabilidad.

Objetivos

- Comprender por qué organizamos un programa.
- Diferenciar funciones y procedimientos.
- Comprender el uso de parámetros.
- Reorganizar el Sistema de Control de Parqueos.

¿Por qué necesitamos organizar nuestros programas?

Antes de escribir una sola línea de código, observa nuevamente el programa de la clase anterior.

 Pregunta para discutir

¿Les parece fácil leer un programa donde todo está escrito dentro del `main()`?

Permite que los estudiantes expresen sus ideas y luego introduce el concepto de modularidad.

La analogía de la construcción de una casa

Una casa no es construida por una sola persona.

Profesional	Responsabilidad
Arquitecto	Diseñar la casa
Albañil	Construir la estructura
Electricista	Instalar el sistema eléctrico
Plomero	Instalar las tuberías
Pintor	Realizar el acabado final

Cada uno tiene una única responsabilidad.

Un programa bien diseñado funciona exactamente igual: cada módulo realiza una tarea específica.

Procedimientos y funciones

¿Qué es un procedimiento?

Un procedimiento es un bloque de código que realiza una tarea específica y **no devuelve un valor**.

Las siguientes tareas de nuestro sistema pueden convertirse en procedimientos:

- Mostrar el título.
- Leer información.
- Mostrar resultados.

i Recuerda

Un procedimiento ejecuta acciones.

¿Qué es una función?

Algunas tareas necesitan calcular un resultado.

Por ejemplo:

- Determinar la tarifa.

- Calcular el total a pagar.

En estos casos utilizaremos una función.

Una función realiza una tarea específica y devuelve un valor al programa que la llamó.

Procedimiento o función

Característica	Procedimiento	Función
Realiza una tarea		
Devuelve un valor		
Utiliza return	No	Sí

Reflexiona

¿Cuáles tareas del Sistema de Control de Parques implementarías mediante procedimientos y cuáles mediante funciones?

¿Qué son los parámetros?

Una función sabe **qué debe hacer**, pero necesita información para poder trabajar.

A esa información la llamamos **parámetros**.

La analogía de la casa

Cada trabajador necesita materiales para realizar su trabajo.

Profesional	¿Qué necesita?
Electricista	Cables e interruptores
Albañil	Ladrillos y cemento
Pintor	Pintura

De la misma manera, una función necesita recibir datos para realizar su tarea.

Un parámetro es un dato que se envía a una función o procedimiento para que pueda realizar su trabajo.

Idea importante

Las funciones siempre realizan la misma tarea.
Lo único que cambia son los datos que reciben.

La siguiente sección transformará paso a paso nuestro Sistema de Control de Parques utilizando procedimientos, funciones y parámetros.

Transformando nuestro programa

Hasta este momento ya conocemos los conceptos de **procedimientos**, **funciones** y **parámetros**.

Ahora llegó el momento de aplicarlos sobre el programa desarrollado en la clase anterior.

Nuestro objetivo **no es cambiar el funcionamiento del Sistema de Control de Parques**.

El objetivo es **organizar mejor el código**.

Al finalizar esta sección, el programa realizará exactamente las mismas tareas, pero será mucho más fácil de leer, comprender y mantener.

Primera mejora: mover el título del programa

Objetivo

Crear un procedimiento que sea el único responsable de mostrar el encabezado del sistema.

Piensa

Si mañana deseamos cambiar el nombre del sistema o mejorar el diseño del encabezado, ¿en cuántos lugares tendríamos que modificar el programa?

Después de crear el procedimiento, elimina ese código del `main()` y reemplázalo por una única llamada al procedimiento.

Analiza el resultado.

El programa continúa funcionando exactamente igual.
Sin embargo, el `main()` ya es más corto y fácil de leer.

Segunda mejora: separar la lectura de datos

Objetivo

Crear un procedimiento encargado de solicitar toda la información al usuario.
Este procedimiento será responsable de:

- Leer el nombre del cliente.
- Leer el tipo de vehículo.
- Leer la cantidad de horas.

Una vez terminado, el `main()` únicamente deberá llamar al procedimiento.
Observa cómo comienza a desaparecer el código repetitivo del programa principal.

Tercera mejora: crear una función para determinar la tarifa

Hasta este momento el programa determina la tarifa utilizando una estructura de decisión.
Ahora esa responsabilidad será trasladada a una función.

Objetivo

Crear una función que reciba el tipo de vehículo y devuelva la tarifa correspondiente.
El programa principal ya no necesitará conocer cómo se calcula la tarifa.
Únicamente solicitará el resultado.

Importante

Una buena función oculta los detalles del cálculo.
El programa principal solamente utiliza el resultado obtenido.

Cuarta mejora: crear una función para calcular el total

Una vez conocida la tarifa, podemos crear otra función cuya única responsabilidad sea calcular el total a pagar.

Esta función recibirá:

- La tarifa.
- La cantidad de horas.

Y devolverá el monto total.

Observa que ahora cada función tiene una única responsabilidad.

Quinta mejora: crear un procedimiento para mostrar el reporte

Finalmente trasladaremos toda la salida del programa a un procedimiento.

Este procedimiento mostrará:

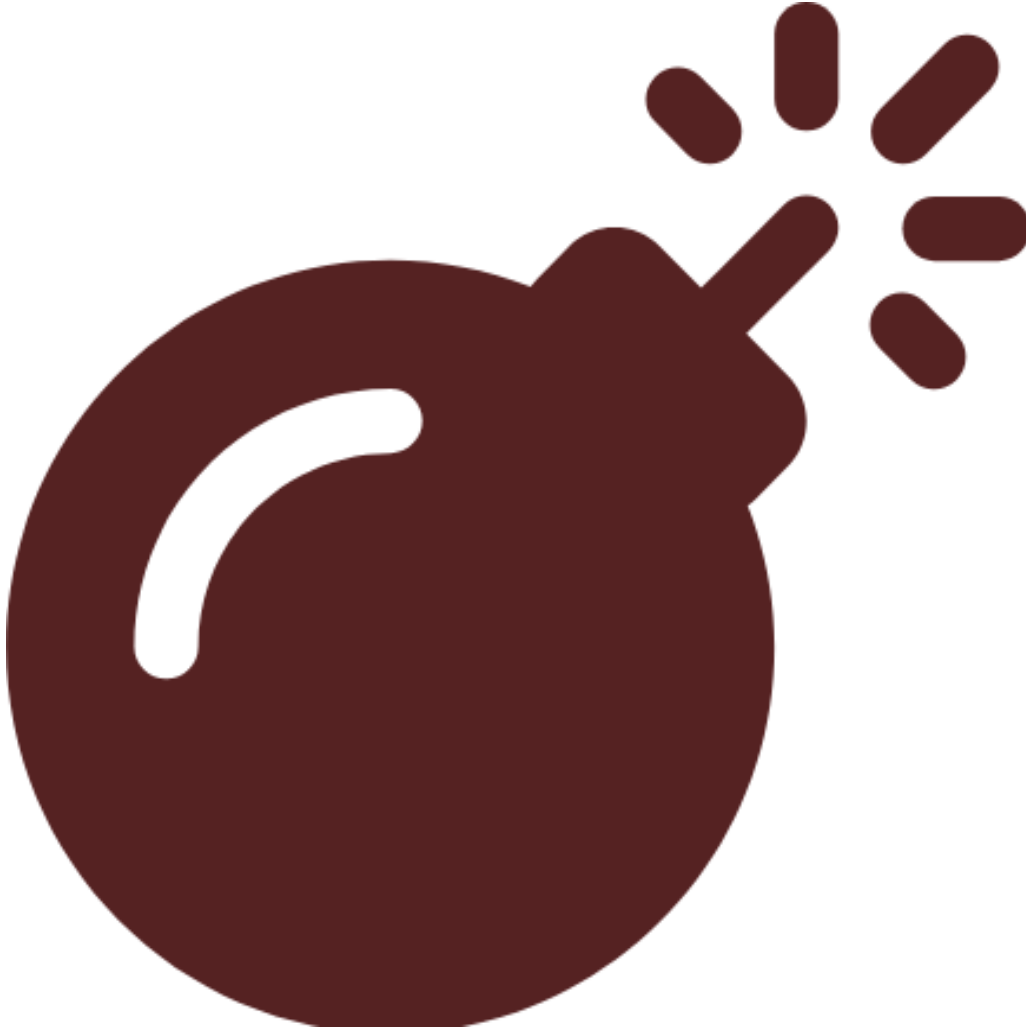
- Nombre del cliente.
- Tipo de vehículo.
- Horas.
- Tarifa.
- Total a pagar.

El programa principal dejará de preocuparse por la presentación de la información.

Su única responsabilidad será coordinar la ejecución del programa.

Nuestro nuevo programa

Al finalizar las mejoras, el `main()` será mucho más pequeño y mucho más fácil de comprender. Su lectura se parece más a una lista de tareas que a un bloque de instrucciones.



💡 Reflexiona

Observa el diagrama anterior.
Cada bloque realiza una única responsabilidad.
Esta es una de las principales características de un programa bien organizado.

¿Qué ganamos con esta organización?

Al reorganizar nuestro programa obtenemos varias ventajas:

- El código es más fácil de leer.
- Cada módulo tiene una responsabilidad específica.
- Las modificaciones futuras serán más sencillas.
- Es posible reutilizar funciones en otros programas.
- El trabajo en equipo resulta más simple, ya que diferentes programadores pueden trabajar sobre módulos distintos.

En el siguiente apartado compararemos la primera versión del programa con esta nueva versión organizada para identificar las mejoras obtenidas.

Primera modificación: crear el procedimiento `mostrarTitulo()`

Hasta este momento, el encabezado del sistema se encuentra escrito directamente dentro de `main()`.

El primer cambio será mover únicamente esas instrucciones a un procedimiento independiente.

1. Identificamos el código que vamos a mover

Localiza dentro de `main()` las siguientes instrucciones:

```
cout << "\n=====\\n";  
cout << "    SISTEMA DE CONTROL DE PARQUEOS\\n";  
cout << "=====\\n\\n";
```

Estas líneas cumplen una sola responsabilidad: mostrar el título del sistema.

2. Creamos el procedimiento

Antes de `main()`, escribe:

```
//-----
// Muestra el encabezado del sistema
//-----
void mostrarTitulo()
{
    cout << "\n=====\\n";
    cout << "    SISTEMA DE CONTROL DE PARQUEOS\\n";
    cout << "=====\\n\\n";
}
```

La palabra void indica que el procedimiento realiza una acción, pero no devuelve ningún valor.

3. Modificamos main()

Elimina del main() las tres instrucciones originales y reemplázalas por:

```
mostrarTitulo();
```

4. Código después de la primera modificación

```
#include <iostream>
#include <string>

using namespace std;

//-----
// Enumeración de los tipos de vehículos
//-----
enum TipoVehiculo
{
    MOTO = 1,
    CARRO,
    CAMIONETA
};

//-----
// Declaración de variables
```

```

//-----
string nombreConductor;
string placa;

int tipoVehiculo;
int horas;

double tarifa = 0.0;
double totalPagar = 0.0;

//-----
// Muestra el encabezado del sistema
//-----
void mostrarTitulo()
{
    cout << "\n=====\\n";
    cout << "    SISTEMA DE CONTROL DE PARQUEOS\\n";
    cout << "=====\\n\\n";
}

int main()
{
    cout << "Nombre: ";
    getline(cin, nombreConductor);

    cout << "Placa: ";
    getline(cin, placa);

    cout << "Horas: ";
    cin >> horas;

    cout << "\\nTipo de vehículo" << endl;
    cout << "1. Moto" << endl;
    cout << "2. Carro" << endl;
    cout << "3. Camioneta" << endl;
    cout << "\\nSeleccione una opción: ";
    cin >> tipoVehiculo;

    switch (tipoVehiculo)
    {
        case MOTO:
            tarifa = 10;

```

```

        break;

    case CARRO:
        tarifa = 15;
        break;

    case CAMIONETA:
        tarifa = 25;
        break;

    default:
        tarifa = 0;
}

totalPagar = tarifa * horas;

mostrarTitulo();

cout << "Conductor : " << nombreConductor << endl;
cout << "Placa      : " << placa << endl;

cout << "Vehículo  : ";

switch (tipoVehiculo)
{
    case MOTO:
        cout << "Moto";
        break;

    case CARRO:
        cout << "Carro";
        break;

    case CAMIONETA:
        cout << "Camioneta";
        break;

    default:
        cout << "No válido";
}

cout << endl;

```

```

    cout << "Horas      : " << horas << endl;
    cout << "Tarifa      : Q" << tarifa << endl;
    cout << "-----" << endl;
    cout << "TOTAL A PAGAR: Q" << totalPagar << endl;

    return 0;
}

```

💡 Comprobación

Compila y ejecuta el programa.
La salida debe ser exactamente la misma que antes.

i ¿Qué cambió?

El encabezado ya no está escrito directamente dentro de `main()`.
Ahora esa tarea pertenece al procedimiento `mostrarTitulo()`.

Segunda modificación: crear el procedimiento `leerDatos()`

En la modificación anterior organizamos el encabezado del sistema.

Ahora haremos lo mismo con la entrada de datos.

Nuestro objetivo será que **todo el proceso de captura de información** quede dentro de un solo procedimiento.

1. Identificamos el código que vamos a mover

Dentro de `main()` localiza todas las instrucciones utilizadas para leer:

- Nombre del conductor.
- Placa.
- Horas.
- Tipo de vehículo.

Todo ese bloque pertenece a una misma responsabilidad: **leer los datos del usuario**.

2. Creamos el procedimiento

Antes de `main()` agrega el siguiente procedimiento:

```
//-----  
// Solicita la información del conductor  
//-----  
void leerDatos()  
{  
    cout << "Nombre: ";  
    getline(cin, nombreConductor);  
  
    cout << "Placa: ";  
    getline(cin, placa);  
  
    cout << "Horas: ";  
    cin >> horas;  
  
    cout << "\nTipo de vehículo" << endl;  
    cout << "1. Moto" << endl;  
    cout << "2. Carro" << endl;  
    cout << "3. Camioneta" << endl;  
    cout << "\nSeleccione una opción: ";  
    cin >> tipoVehiculo;  
}
```

Observa que el procedimiento únicamente realiza una tarea: solicitar información al usuario.

3. Modificamos `main()`

Ahora elimina todas las instrucciones de entrada de datos y reemplázalas por una sola línea:

```
leerDatos();
```

El inicio del programa principal ahora queda mucho más limpio.

```
int main()  
{  
    leerDatos();  
}
```

```
} // El resto del programa continúa aquí...
```

4. ¿Qué ganamos?

Comparemos ambas versiones.

Antes

```
main()
    Leer nombre
    Leer placa
    Leer horas
    Mostrar menú
    Leer tipo de vehículo
    ...
```

Después

```
main()
    leerDatos()
    ...
```

El programa sigue haciendo exactamente lo mismo, pero ahora el nombre del procedimiento describe claramente la tarea que realiza.

Comprobación

Compila nuevamente el programa.
Si todo está correcto, el comportamiento será exactamente el mismo que antes.

¿Qué aprendimos?

Un procedimiento puede agrupar varias instrucciones relacionadas en una sola unidad lógica.
Esto hace que el programa sea más fácil de leer y mantener.

Preparándonos para la siguiente modificación

En la siguiente sección moveremos el `switch` encargado de determinar la tarifa hacia una **función**.

Será la primera vez que nuestro programa devolverá un valor utilizando `return`.

Tercera modificación: crear la función `determinarTarifa()`

Hasta este momento nuestro programa utiliza un bloque `switch` dentro de `main()` para determinar la tarifa según el tipo de vehículo.

En esta modificación moveremos esa responsabilidad hacia una función.

1. Identificamos el código que vamos a mover

Localiza el siguiente bloque dentro de `main()`:

```
switch (tipoVehiculo)
{
    case MOTO:
        tarifa = 10;
        break;

    case CARRO:
        tarifa = 15;
        break;

    case CAMIONETA:
        tarifa = 25;
        break;

    default:
        tarifa = 0;
}
```

Observa que este bloque realiza una única tarea: **determinar la tarifa**.

2. Creamos la función

Antes de `main()` agrega la siguiente función:

```
//-----  
// Determina la tarifa según el tipo de vehículo  
//-----  
double determinarTarifa(int tipo)  
{  
    switch (tipo)  
    {  
        case MOTO:  
            return 10;  
  
        case CARRO:  
            return 15;  
  
        case CAMIONETA:  
            return 25;  
  
        default:  
            return 0;  
    }  
}
```

Analicemos el encabezado

```
double determinarTarifa(int tipo)
```

Elemento	Significado
double	La función devolverá un valor decimal.
determinarTarifa	Nombre de la función.
int tipo	Parámetro que recibe el tipo de vehículo.

A diferencia de un procedimiento, esta función **devuelve un resultado** mediante la instrucción `return`.

3. Modificamos `main()`

Elimina el bloque `switch` y reemplázalo por una única línea:

```
tarifa = determinarTarifa(tipoVehiculo);
```

Lee esa instrucción en voz alta:

“La variable **tarifa** recibirá el valor que devuelva la función **determinarTarifa()**.”

4. ¿Qué sucede internamente?

```
main()
    envía tipoVehiculo
determinarTarifa()
    calcula la tarifa
return tarifa
main() guarda el resultado en la variable tarifa
```



5. ¿Qué ganamos?

Antes

```
main()
    switch(...)
    case MOTO
    case CARRO
    case CAMIONETA
```

Después

```
main()
    tarifa = determinarTarifa(tipoVehiculo);
```

Ahora el programa principal ya no necesita conocer cómo se calcula la tarifa.

Esa responsabilidad quedó encapsulada dentro de la función.

Comprobación

Compila y ejecuta nuevamente el programa.
El resultado debe ser exactamente el mismo que en las modificaciones anteriores.

¿Qué aprendimos?

Una función permite ocultar los detalles de un cálculo.
El programa principal únicamente solicita el resultado y continúa con la siguiente tarea.

Preparándonos para la siguiente modificación

En la siguiente modificación construiremos la función `calcularTotal()`.

Verás que el `main()` comienza a parecer una lista de tareas de alto nivel.

Cuarta modificación: crear la función `calcularTotal()`

En la modificación anterior logramos que el programa determinara la tarifa utilizando una función.

Ahora haremos exactamente lo mismo con el cálculo del total a pagar.

1. Identificamos el código que vamos a mover

Dentro de `main()` localiza la siguiente instrucción:

```
totalPagar = tarifa * horas;
```

Aunque únicamente es una línea de código, representa una responsabilidad muy importante del programa: **calcular el monto que deberá pagar el cliente.**

2. Creamos la función

Antes de `main()` agrega la siguiente función:

```
//-----  
// Calcula el total a pagar  
//-----  
double calcularTotal(double tarifaHora, int cantidadHoras)  
{  
    return tarifaHora * cantidadHoras;  
}
```

Observa que esta función recibe dos parámetros:

Parámetro	Descripción
<code>tarifaHora</code>	Tarifa correspondiente al vehículo
<code>cantidadHoras</code>	Horas que permaneció estacionado

Después realiza el cálculo y devuelve el resultado mediante `return`.

3. Modificamos `main()`

Elimina la instrucción original y reemplázala por:

```
totalPagar = calcularTotal(tarifa, horas);
```

Lee la instrucción como si fuera una frase:

“TotalPagar recibirá el valor que devuelva la función `calcularTotal()` utilizando la tarifa y las horas.”

4. ¿Qué sucede internamente?

```
main()
    envía tarifa y horas
calcularTotal()
    realiza el cálculo
return total
main() guarda el resultado
```



5. ¿Qué ganamos?

Antes

```
main()
    tarifa = determinarTarifa(...)
    totalPagar = tarifa * horas
    mostrar resultados
```

Después

```
main()
    tarifa = determinarTarifa(...)
    totalPagar = calcularTotal(...)
    mostrar resultados
```

Ahora el programa principal ya no realiza cálculos directamente.

Únicamente coordina la ejecución de las funciones.

Comprobación

Compila nuevamente el programa.

La salida debe ser exactamente igual que en las versiones anteriores.

¿Qué aprendimos?

Cada función debe tener una única responsabilidad.

`determinarTarifa()` determina la tarifa.

`calcularTotal()` calcula el total.

Separar responsabilidades hace que el programa sea más fácil de comprender y mantener.

Preparándonos para la siguiente modificación

Solo nos queda un paso.

Moveremos toda la presentación de resultados al procedimiento `mostrarResultados()`.

Cuando terminemos, el `main()` quedará reducido a unas pocas líneas y describirá claramente el flujo del programa.

Quinta modificación: crear el procedimiento `mostrarResultados()`

Ya organizamos la entrada de datos y los cálculos del programa.

Solo nos queda una última responsabilidad: presentar la información al usuario.

En esta modificación moveremos todo el reporte final hacia un procedimiento independiente.

1. Identificamos el código que vamos a mover

Localiza dentro de `main()` todas las instrucciones utilizadas para mostrar:

- El encabezado.
- El nombre del conductor.
- La placa.
- El tipo de vehículo.

- Las horas.
- La tarifa.
- El total a pagar.

Todo ese bloque tiene una única responsabilidad: **mostrar el resultado del proceso.**

2. Creamos el procedimiento

Antes de `main()` agrega el siguiente procedimiento.

```
//-----  
// Muestra el reporte final  
//-----  
void mostrarResultados()  
{  
    mostrarTitulo();  
  
    cout << "Conductor : " << nombreConductor << endl;  
    cout << "Placa      : " << placa << endl;  
  
    cout << "Vehículo  : ";  
  
    switch (tipoVehiculo)  
    {  
        case MOTO:  
            cout << "Moto";  
            break;  
  
        case CARRO:  
            cout << "Carro";  
            break;  
  
        case CAMIONETA:  
            cout << "Camioneta";  
            break;  
  
        default:  
            cout << "No válido";  
    }  
  
    cout << endl;
```

```

    cout << "Horas      : " << horas << endl;
    cout << "Tarifa      : Q" << tarifa << endl;
    cout << "-----" << endl;
    cout << "TOTAL A PAGAR: Q" << totalPagar << endl;
}

```

Observa algo interesante.

Dentro de este procedimiento estamos llamando nuevamente a:

```
mostrarTitulo();
```

Esto demuestra que un procedimiento puede utilizar otro procedimiento.

3. Modificamos main()

Elimina todo el bloque de salida y reemplázalo por una sola instrucción.

```
mostrarResultados();
```

4. Nuestro nuevo main()

Después de todas las modificaciones, el programa principal queda reducido a unas pocas líneas.

```

int main()
{
    leerDatos();

    tarifa = determinarTarifa(tipoVehiculo);

    totalPagar = calcularTotal(tarifa, horas);

    mostrarResultados();

    return 0;
}

```

Ahora el main() puede leerse casi como un algoritmo escrito en lenguaje natural.

1. Leer datos.
2. Determinar la tarifa.
3. Calcular el total.
4. Mostrar resultados.

5. ¿Qué cambió?

Comparemos ambas versiones.

Antes

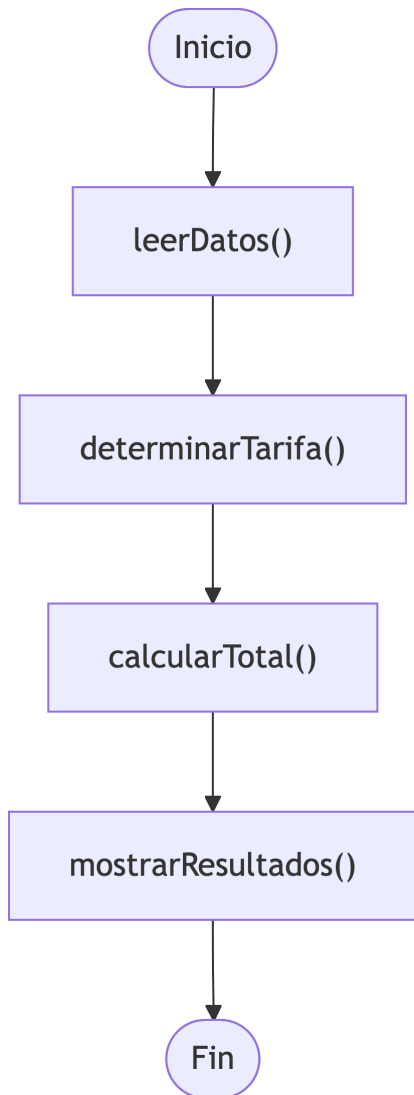
```
main()
  Leer datos
  Determinar tarifa
  Calcular total
  Mostrar encabezado
  Mostrar conductor
  Mostrar vehículo
  Mostrar horas
  Mostrar tarifa
  Mostrar total
```

Después

```
main()
  leerDatos()
  determinarTarifa()
  calcularTotal()
  mostrarResultados()
```

Observa que el programa realiza exactamente las mismas tareas.

La diferencia es que ahora cada responsabilidad fue trasladada a un módulo independiente.



Comprobación final

Compila nuevamente el programa.

Utiliza los mismos datos empleados en el capítulo anterior.

El resultado obtenido debe ser exactamente el mismo.

Si el comportamiento no cambió, significa que la reorganización fue realizada correctamente.

Felicitaciones

Has transformado un programa funcional en un programa mucho mejor organizado. Eso es precisamente lo que ocurre durante el desarrollo profesional del software.

Lo que aprendimos

Durante este capítulo reorganizamos un programa completo utilizando procedimientos, funciones y parámetros.

Responsabilidad	Módulo
Mostrar el encabezado	<code>mostrarTitulo()</code>
Leer información	<code>leerDatos()</code>
Determinar la tarifa	<code>determinarTarifa()</code>
Calcular el total	<code>calcularTotal()</code>
Mostrar resultados	<code>mostrarResultados()</code>

Observa que cada módulo tiene una única responsabilidad.

Gracias a ello, el programa principal quedó reducido a unas pocas líneas fáciles de leer.

Idea clave

Programar mejor no significa escribir más código.
Programar mejor significa organizar el código de manera que cualquier persona pueda comprenderlo y mantenerlo.

Preparándonos para el siguiente capítulo

Nuestro Sistema de Control de Parqueos ya está organizado.

En el siguiente capítulo comenzaremos a trabajar con **arreglos**, lo que permitirá registrar y administrar la información de varios vehículos en lugar de procesar únicamente uno.